

Fundamentos

Programação Funcional

Marco A L Barbosa

malbarbo.pro.br

Departamento de Informática

Universidade Estadual de Maringá



Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-Compartilhagual 4.0 Internacional.

<http://github.com/malbarbo/na-progfun>

Introdução

O paradigma de programação funcional é baseado na definição e aplicação de funções.

Uma **função** é um mapeamento de valores de entrada para valores de saída, definido por meio de expressões.

Uma **expressão** é uma entidade sintática que quando avaliada (reduzida) produz um valor.

Executar um programa funcional é avaliar expressões: aplicar as regras de avaliação até chegar a um valor (nos paradigmas imperativos, executar é seguir uma sequência de comandos que alteram o estado).

Vamos ver uma sequência de definições de expressões e regras de avaliação.

Uma **expressão** consiste em

- Um literal; ou
- Uma função primitiva

Um **literal** é a representação direta de um valor no código.

Uma **função primitiva** é uma função suportada diretamente pela linguagem de programação.

Um **tipo primitivo** é um tipo suportado diretamente pela linguagem de programação.

Vamos tratar as funções da biblioteca padrão também como primitivas: o que importa aqui é que podemos usá-las sem examinar como são implementadas.

Gleam provê 9 tipos primitivos. Todos os nomes de tipos começam com letra maiúscula.

Números inteiros (`Int`)

- `1345`
- `9_876`

Booleanos (`Bool`)

- `True`
- `False`

Números de ponto flutuante (`Float`)

- `2.65`
- `2.0e12`
- `7.4e-10`

Strings (`String`)

- `"din uem"`
- `"apenas \"um\" teste"`

Veremos alguns outros tipos primitivos ao longo da disciplina.

Funções primitivas

Gleam provê diversas funções primitivas, a maioria delas está disponível na forma de operadores. Todos os nomes de funções começam com letra minúscula.

Operações com inteiros:

- `+` (`int.add`)
- `-` (`int.subtract`)
- `*` `/` `%` `>` `>=` `<` `<=` `==` `!=`
- `int.to_float` e diversas outras no módulo `int`.

Operações com floats:

- `.*` (`float.multiply`)
- `/.` (`float.divide`)
- `+. -.` `>.` `>=.` `<.` `<=.` `==` `!=`
- `float.truncate` e diversas outras no módulo `float`.

Operações com strings:

- `<>` (`string.append`)
- `==` `!=`
- `string.slice` e diversas outras no módulo `string`.

Operações com booleanos:

- `!` (`bool.negate`)
- `==` `!=`
- Outros operadores que veremos depois.

Uma **expressão** consiste em

- Um literal; ou
- Uma função primitiva

Regra para **avaliação de expressão**

- Literal $\rightarrow$ valor que o literal representa
- Função primitiva $\rightarrow$ sequência de instruções de máquina associada com a função

Como a regra de avaliação de expressão está ligada com a definição de expressão?

Uma expressão é definida em termos de dois casos e por isso a regra de avaliação de expressão também é definida por dois casos.

Exemplo de avaliação de expressões

```
> True
```

```
True
```

```
> 231
```

```
231
```

```
> "Banana"
```

```
"Banana"
```

```
> import gleam/int
```

```
> int.add
```

```
//fn(a, b) { ... }
```

As funções da biblioteca padrão, como `int.add`, ficam em módulos e precisam ser importadas antes de serem usadas. Para não poluir os exemplos, os `import` são omitidos daqui em diante, mas eles continuam necessários no REPL e nos arquivos.

Essa definição de expressão parece bastante limitada. O que está faltando? Uma forma de combinar expressões para formar novas expressões!

Combinações

Alguns exemplos de combinações. (Observe que em Gleam as chaves { } são usadas para agrupar expressões — os parênteses () são usados apenas nas chamadas de funções)

```
> { 2 + 12 } * 5
```

```
70
```

```
> "Gol" <> string.repeat("!", 4)
```

```
"Gol!!!!"
```

```
> 10 / 0 // Divisão por zero é zero!
```

```
0
```

```
> int.multiply(int.add(2, 12), 5)
```

```
70
```

```
> string.append("Gol",  
                string.repeat("!", 4))
```

```
"Gol!!!!"
```

```
> int.divide(10, 0)
```

```
Error(Nil)
```

O terceiro par não é uma equivalência: `int.divide` sinaliza o erro na resposta, usando um tipo que veremos no capítulo **Funções totais**.

Considerando apenas funções e literais (vamos deixar os operadores de lado por simplicidade), qual é a forma de combinar expressões para criar novas expressões?

A chamada de função. Como podemos definir como são formadas as chamadas de funções?

Primeira tentativa

Uma chamada de função começa com uma **função primitiva**, seguida de abre parêntese, seguido de uma sequência de **literais** separados por vírgula, seguida de fecha parêntese.

A definição é adequada? Não! Ela não contempla `int.multiply(int.add(2, 12), 5)`!

Segunda tentativa

Uma chamada de função começa com uma **função primitiva**, seguida de abre parêntese, seguido de uma sequência de **expressões** separadas por vírgula, seguida de fecha parêntese.

Repare que generalizamos literais para expressões mas não generalizamos funções primitivas. Vamos fazer isso! Essa generalização pode parecer estranha ou até inútil, mas vamos apreciar a sua utilidade ao longo da disciplina.

Uma **chamada de função** consiste em uma **expressão** seguida por uma sequência de **expressões** entre parênteses separadas por vírgula.

- A primeira expressão é o **operador**;
- As demais expressões são os **operandos**.

Qual é o valor produzido pela avaliação de uma chamada de função?

- O resultado da aplicação do valor do operador aos valores dos operandos.

Além de um valor, toda expressão tem um **tipo** — e o tipo é determinado pelo compilador **antes** da avaliação, sem executar o programa.

O tipo de um literal é o tipo do valor que ele representa; o tipo de uma função descreve as entradas e a saída (`int.add` tem tipo `fn(Int, Int) -> Int`).

E o tipo de uma chamada de função? É o tipo da saída da função: `int.add(2, 3)` tem tipo `Int`.

Os tipos também determinam quais chamadas são válidas: o operador precisa ser uma função e cada operando precisa ter o tipo da entrada correspondente. `int.add(2, "12")` tem a forma de uma chamada, mas nem chega a ser avaliada: o compilador a rejeita.

E os operadores que deixamos de lado?

Uma expressão com operador binário é uma chamada de função escrita de outra forma: em `int.add(2, 3)` o operador vem **antes** dos operandos (forma **prefixa**), em `2 + 3` ele vem **entre** eles (forma **infixa**). A regra de avaliação é a mesma.

Em Gleam a forma infix é apenas uma notação: `+` sozinho não é uma expressão, não é possível escrever `+(2, 3)` nem usar `+` como argumento. Por isso a lista de operações primitivas mostra, ao lado de cada operador, o nome da função correspondente.

Na forma prefixa os parênteses deixam a estrutura explícita; na forma infixa ela depende das regras de prioridade.

Entre os operadores aritméticos, as prioridades são as que já estamos acostumados da matemática: $2 + 3 * 5$ é `int.add(2, int.multiply(3, 5))`, e não `int.multiply(int.add(2, 3), 5)`.

Os operadores aritméticos têm prioridade maior que os relacionais: $1 + 2 == 3$ é `{ 1 + 2 } == 3`.

Use `{ }` quando quiser uma estrutura diferente da que as prioridades determinam.

Vamos atualizar a definição de expressão para incluir as chamadas de funções.

Uma **expressão** consiste em

- Um literal; ou
- Uma função primitiva; ou
- Uma chamada de função (**expressão** seguida de uma lista de **expressões** entre parênteses)

Regra para **avaliação de expressão**

- Literal → valor que o literal representa
- Função primitiva → sequência de instruções de máquina associada com a função
- Chamada de função
 - **Avalie cada expressão** da chamada da função, isto é, reduza cada expressão para um valor
→ resultado da aplicação do operador aos operandos

Algumas observações interessantes

- Quando uma expressão é uma chamada de função, ela contém outras expressões. Quando uma definição refere-se a si mesma, dizemos que ela é uma definição com **autorreferência**. O uso de autorreferência permite a criação de expressões de tamanhos arbitrários.
- O processo de avaliação para uma expressão que é uma chamada de função requer a chamada do processo de avaliação para suas expressões. Quando um processo é definido em termos de si mesmo, dizemos que ele é **recursivo**. O uso de recursividade permite a avaliação de expressões de tamanhos arbitrários.
- Uma autorreferência na definição implica (geralmente) uma recursão no processo que a percorre. Estamos usando esses conceitos para entender a estrutura da linguagem Gleam, mas eles também são fundamentais para criar programas no paradigma funcional, como veremos ao projetar funções para dados autorreferentes.

Avaliação de expressões

Exemplo de avaliação de uma expressão

```
import gleam/int.{add, multiply as mul, subtract as sub}
```

```
3 * { 2 * 4 + { 3 + 5 } } + { { 10 - 7 } + 6 }
```

```
add(mul(3, add(mul(2, 4), add(3, 5))), add(sub(10, 7), 6)) // mul(2, 4) -> 8
```

```
add(mul(3, add(8, add(3, 5))), add(sub(10, 7), 6)) // add(3, 5) -> 8
```

```
add(mul(3, add(8, 8)), add(sub(10, 7), 6)) // add(8, 8) -> 16
```

```
add(mul(3, 16), add(sub(10, 7), 6)) // mul(3, 16) -> 48
```

```
add(48, add(sub(10, 7), 6)) // sub(10, 7) -> 3
```

```
add(48, add(3, 6)) // add(3, 6) -> 9
```

```
add(48, 9) // add(48, 9) -> 57
```

57

Definições

Vimos anteriormente que o paradigma de programação funcional é baseado na definição e aplicação de funções.

Como funções são definidas em termos de expressões, nós vimos primeiramente o que são expressões.

Agora vamos ver o que são definições.

Qual é o propósito das definições?

Definições servem para dar nome a objetos computacionais, sejam dados ou funções.

- É a forma de abstração mais elementar

Definições de constantes

A forma geral para definições de constantes em Gleam é:

```
[pub] const nome [: Tipo] = literal
```

O **pub** é opcional: ele torna a definição visível para outros módulos. Sem ele, a definição só pode ser usada dentro do módulo em que aparece, e nem a janela de interações a enxerga. Assim como os **import**, o **pub** é omitido nos slides daqui em diante para economizar espaço, mas ele continua necessário nos arquivos.

```
const x: Int = 10      > x
pub const y = 20       10
                        > y
                        20
```

Note que a especificação do tipo da constante é opcional. Se o tipo não for especificado, ele é inferido pelo compilador.

Definições de funções

A forma geral para funções definidas pelo programador (**funções compostas**) em Gleam é:

```
[pub] fn nome(parametro1 [: Tipo], parametro2 [: Tipo], ...) [-> Tipo] {  
  expressão...  
}
```

Exemplos

<pre>fn quadrado(x: Int) -> Int { x * x }</pre>	<pre>> quadrado(6) 36</pre>
<pre>fn soma_quadrados(a: Int, b) { quadrado(a) + quadrado(b) }</pre>	<pre>> soma_quadrados(3, 4) 25</pre>

Note que a especificação dos tipos das entradas e saída é opcional. Se os tipos não forem especificados, eles são inferidos pelo compilador.

Os nomes usados nas definições são associados com os objetos que eles representam e armazenados em uma memória chamada de **ambiente**.

Um programa em Gleam é composto por uma sequência de instruções **import** e de definições.

Agora precisamos estender a definição de expressões para incluir nomes e alterar a regra de avaliação de expressões para considerar a chamada de funções compostas.

Modelo de substituição

Uma **expressão** consiste em

- Um literal; ou
- Uma função primitiva; ou
- Um nome; ou
- Uma chamada de função (**expressão** seguida de uma lista de **expressões** entre parênteses)

Regra para **avaliação de expressão**

- Literal $\rightarrow$ valor que o literal representa
- Função primitiva $\rightarrow$ sequência de instruções de máquina associada com a função
- Nome $\rightarrow$ valor associado com o nome no ambiente
- Chamada de função
 - **Avalie cada expressão** da chamada da função
 - Se o operador é uma função primitiva, aplique a função aos argumentos
 - Senão (o operador é uma função composta) , **avalie** o corpo da função **substituindo** cada ocorrência do parâmetro formal pelo argumento correspondente (**modelo de substituição**)

Modelo de substituição

```
fn quadrado(x) { x * x }
fn soma_quadrados(a, b) { quadrado(a) + quadrado(b) }
fn f(a) { soma_quadrados(a + 1, a * 2) }

f(5)                                // Substitui f(5) pelo corpo de f com as ocorrências
                                    // do parâmetro a substituídas pelo argumento 5

soma_quadrados(5 + 1, 5 * 2) // Reduz 5 + 1 para o valor 6

soma_quadrados(6, 5 * 2) // Reduz 5 * 2 para o valor 10

soma_quadrados(6, 10) // Substitui soma_quadrados(6, 10) pelo corpo ...

quadrado(6) + quadrado(10) // Substitui quadrado(6) pelo corpo ...

{ 6 * 6 } + quadrado(10) // Reduz 6 * 6 para 36

36 + quadrado(10) // Substitui quadrado(10) pelo corpo ...

36 + { 10 * 10 } // Reduz 10 * 10 para 100

36 + 100 // Reduz 36 + 100 para 136

136
```

Ao invés de avaliar os operandos e depois fazer a substituição, existe um outro modo de avaliação que primeiro faz a substituição e apenas avalia os operandos quando (e se) eles forem necessários.

```
f(5)
soma_quadrados(5 + 1, 5 * 2)
quadrado(5 + 1) + quadrado(5 * 2)
{ 5 + 1 } * { 5 + 1 } + quadrado(5 * 2)
{ 5 + 1 } * { 5 + 1 } + { 5 * 2 } * { 5 * 2 }
6 * { 5 + 1 } + { 5 * 2 } * { 5 * 2 }
6 * 6 + { 5 * 2 } * { 5 * 2 }
36 + { 5 * 2 } * { 5 * 2 }
36 + 10 * { 5 * 2 }
36 + 10 * 10
36 + 100
136
```

A resposta gerada por essas duas formas de substituição será sempre a mesma?

Para funções que terminam sim!

Veremos depois os casos para funções que não terminam.

O método de avaliação que primeiro avalia os argumentos e depois aplica a função é chamado de **avaliação em ordem aplicativa** (avaliação ansiosa).

O método de avaliação alternativo que primeiro substitui e depois reduz é chamado de **avaliação em ordem normal** (avaliação preguiçosa).

O Gleam usa por padrão a avaliação em ordem aplicativa.

O Haskell usa a avaliação em ordem normal.

Exercícios

O seu amigo Alan está planejando uma viagem para o final do ano com a família e está considerando diversos destinos. Uma das coisas que ele está levando em consideração é o custo da viagem, que inclui, entre outras coisas, hospedagem, combustível e o pedágio. Para o cálculo do combustível ele pediu a sua ajuda, ele disse que sabe a distância que vai percorrer, o preço do litro do combustível e o rendimento do carro (quantos quilômetros o carro anda com um litro de combustível), mas que é muito chato ficar fazendo o cálculo manualmente, então ele quer que você faça um programa para calcular o custo do combustível em uma viagem.

O que de fato precisa ser feito? Calcular o custo do combustível (saída) sabendo a distância do percurso, o preço do litro e o rendimento do carro (entradas).

Como determinar a forma que a saída é computada a partir da entrada? Fazendo exemplos específicos e generalizando.

Exemplo de entrada

- Distância: 400.0 Km
- Preço do litro: R\$ 5.0
- Rendimento: 10.0 Km/l

Saída

- Quantidade de litros (Distância / Rendimento): $400.0 / 10.0 \rightarrow 40.0$
- Custo (Quantidade de litros $\times$ Preço do litro): $40.0 * 5.0 \rightarrow 200.0$

Implementação

```
fn custo_combustivel(distancia, preco_do_litro, rendimento) {  
  { distancia /. rendimento } *. preco_do_litro  
}
```

Verificação

Como verificar se a implementação faz o que é esperado?

Executando os exemplos que fizemos anteriormente:

```
> custo_combustivel(400.0, 5.0, 10.0)  
200.0
```

Definições locais

Nos exemplos, calculamos a resposta em dois passos: primeiro a quantidade de litros, depois o custo. Mas a implementação juntou os dois passos em uma única expressão.

Podemos dar um nome ao resultado do primeiro passo com uma **definição local**:

```
fn custo_combustivel(distancia, preco_do_litro, rendimento) {  
  let litros = distancia /. rendimento  
  litros *. preco_do_litro  
}
```

O corpo de uma função é uma sequência de definições locais seguida de uma expressão; é essa expressão que produz o valor da função.

O **let** apenas associa um nome a um valor, assim como as definições que vimos antes; a diferença é que o nome vale do ponto em que é definido até o final do bloco em que ele aparece.

O `let` tem uma regra de avaliação própria, mas não precisamos inventá-la do zero: podemos traduzir a definição local para algo que já conhecemos, uma chamada de função auxiliar.

```
fn custo_combustivel(distancia, preco_do_litro, rendimento) {  
    custo(distancia /. rendimento, preco_do_litro)  
}
```

```
fn custo(litros, preco_do_litro) {  
    litros *. preco_do_litro  
}
```

Aqui vale o modelo de substituição: `distancia /. rendimento` é avaliado e o valor substitui `litros` no corpo de `custo` — e é isso que o `let` faz.

Em geral, o parâmetro da função auxiliar é o nome definido pelo **let**, o corpo dela é o resto do bloco, e o argumento da chamada é a expressão à direita do **=**:

```
let nome = expressão1      letf(expressão1)
expressão2                  fn letf(nome) {
                              expressão2
                              }
```

Note que os demais nomes que o resto do bloco usa, como **preco_do_litro**, precisariam virar parâmetros adicionais de **letf**, mas no capítulo **Funções como valores** veremos que nem os parâmetros adicionais e nem o nome da função são necessários.

Depois que você fez o programa para o Alan, a Márcia, amiga em comum de vocês, soube que você está oferecendo serviços desse tipo e também quer a sua ajuda. O problema da Márcia é que ela sempre tem que fazer a conta manualmente para saber se deve abastecer o carro com álcool ou gasolina. A conta que ela faz é verificar se o preço do álcool é até 70% do preço da gasolina, se sim, ela abastece o carro com álcool, senão ela abastece o carro com gasolina. Você pode ajudar a Márcia também?

É possível resolver este problema (produzindo como saída o tipo de combustível) usando as coisas que vimos até aqui? Não!

O que está faltando? Algum tipo de expressão condicional.

Voltamos a esse problema no próximo capítulo!

Condicional

A maioria das linguagens tem `if` para expressar seleção, mas a linguagem Gleam não tem. Ao invés do `if`, ela oferece o `case`, que é mais genérico. Vamos supor por um instante que Gleam tivesse `if`.

A forma geral do `if` poderia ser:

```
if expressão { // condição
  expressão...
} else {
  expressão...
}
```

Exemplos

```
> if 4 > 2 { 10 + 2 } else { 7 * 3 }
```

```
12
```

```
> if 10 == 12 { 10 + 2 } else { 7 * 3 }
```

```
21
```

Qual a diferença desse `if` em relação ao das outras linguagens? Esse `if` é uma expressão, ele produz um valor como resultado. Na maioria das outras linguagens o `if` é uma sentença (*statement* em inglês), ele não produz um resultado, ele seleciona quais comandos serão executados, isto é, quais efeitos colaterais serão gerados.

A forma inicial do `case` é:

```
case expressão {      // expressão examinada
  True -> expressão    // caso True
  False -> expressão   // caso False
}
```

Onde a ordem dos casos não é importante.

```
> if 4 > 2 {
  10 + 2
} else {
  7 * 3
}
```

12

```
> case 4 > 2 {
  True -> 10 + 2
  False -> 7 * 3
}
```

12

Regra de avaliação do case

O **case** não é uma chamada de função: ele tem uma regra de avaliação própria. As construções assim são chamadas de **formas especiais**.

A regra de avaliação de expressões **case** é:

- Avalie a expressão examinada
- Se o valor da expressão examinada for **True**, substitua toda a expressão **case** pela expressão do caso **True**
- Senão, substitua toda a expressão **case** pela expressão do caso **False**

Os dois casos precisam ter o mesmo tipo, que é o tipo da expressão **case**.

O **let** que vimos antes também é uma forma especial. Vamos ver outras ao longo da disciplina, como **&&**, **||**, funções anônimas, **|>** e **use**.

Vamos escrever uma função para calcular o valor absoluto de um número, isto é

$$\text{abs}(x) = \begin{cases} x & \text{se } x \geq 0 \\ -x & \text{caso contrário} \end{cases}$$

e ver o processo de avaliação dessa função.

```
fn abs(x) {  
  case x >= 0 {  
    True -> x  
    False -> -x  
  }  
}
```

```
abs(-4)           // Substitui abs(-4) pelo corpo ...  
  
case -4 >= 0 {     // A expressão examinada é avaliada  
  True -> -4  
  False -> - { -4 }  
}  
  
case False {       // Como a expressão examinada é False  
  True -> -4        // o case é substituído pela expressão  
  False -> - { -4 } // do caso False  
}  
  
- { -4 }           // Reduz - { -4 } para 4  
4
```

Uma **expressão** consiste em

- Um literal; ou
- Uma função primitiva; ou
- Um nome; ou
- Uma forma especial; ou
- Uma chamada de função

Regra para **avaliação de expressão**

- Literal → valor que o literal representa
- Função primitiva → sequência de instruções de máquina associada com a função
- Nome → valor associado com o nome no ambiente
- Forma especial → avalie usando a regra de avaliação própria daquela forma (para o **case**, a regra que acabamos de ver)
- Chamada de função → avalie usando a regra de avaliação de chamadas de funções

Defina a função `sinal` que determina o sinal de um número inteiro.

```
> sinal(4)
```

```
1
```

```
> sinal(0)
```

```
0
```

```
> sinal(-7)
```

```
-1
```

```
fn sinal(x) {  
  case x > 0 {  
    True -> 1  
    False ->  
      case x == 0 {  
        True -> 0  
        False -> -1  
      }  
  }  
}
```

Exercício and

Defina a função **and** que recebe os argumentos booleanos **x** e **y** e produz como resposta o E lógico entre eles, isto é

```
> and(False, False)
False
> and(False, True)
False
> and(True, False)
False
> and(True, True)
True
```

Observe os exemplos e responda: se verificarmos o valor de **x**, e ele for **False**, qual é o resultado da função? **False**. E se **x** for **True**? Verificamos o valor de **y**, se for **True**, o resultado da função é **True**, senão, o resultado da função é **False**.

```
fn and(x, y) {
  case x {
    False -> False
    True ->
      case y {
        True -> True
        False -> False
      }
  }
}
```

```
fn and(x, y) {  
  case x {  
    False -> False  
    True ->  
      case y {  
        True -> True  
        False -> False  
      }  
  }  
}
```

Podemos simplificar a função? Sim!

```
fn and(x, y) {  
  case x {  
    False -> False  
    True -> y  
  }  
}
```

E a função **or**, o OU lógico entre **x** e **y**?

```
fn or(x, y) {  
  case x {  
    True -> True  
    False -> y  
  }  
}
```


Limitações das funções `and` e `or`

Existe alguma implicação em definirmos `and` e `or` como funções?

Sim. Na ordem aplicativa todos os argumentos são avaliados antes da chamada, então é impossível deixar de avaliar o segundo argumento.

Mas no `and`, se o primeiro argumento for `False`, não é necessário avaliar o segundo; no `or`, o mesmo vale se o primeiro for `True`. Deixar de avaliar o segundo operando quando o primeiro já determina o resultado é chamado de **avaliação em curto-circuito**, um tipo de avaliação preguiçosa.

Note que a limitação é da ordem de avaliação, não do fato de serem funções: em uma linguagem com avaliação em ordem normal, como Haskell, essa mesma definição de `and` já teria curto-circuito.

Em Gleam, para ter curto-circuito, `&&` e `||` precisam ser formas especiais, com regra de avaliação própria, como veremos a seguir.

Operadores lógicos

A linguagem Gleam oferece os operadores lógicos `&&` (e), `||` (ou) e `!` (negação), com o mesmo significado dos operadores `and`, `or` e `not` do Python.

Os operadores `&&` e `||` são binários e `!` é unário.

Diferente do Python, os operandos precisam ser booleanos: `1 && True` é um erro de tipo.

Exemplos dos operadores lógicos

Alguns exemplos

```
> 3 > 4 || 2 == 1 + 1
```

True

```
> 3 > 4 && 2 == 1 + 1
```

False

Diferente do Python, a negação tem prioridade maior que os operadores relacionais.

```
> ! 3 > 4
```

error: Type mismatch

```
> !{ 3 > 4 }
```

True

```
> !True
```

False

```
> bool.negate(2 == 4)
```

True

Os operadores relacionais têm prioridade maior que `&&` e `||`.

O operador `&&` tem maior prioridade do que `||`.

```
> True || False && False
```

True

```
> { True || False } && False
```

False

Assim como em outras linguagens, os operadores `&&` e `||` são avaliados em curto-circuito.

Ou seja, `&&` e `||` são formas especiais: têm regras de avaliação próprias e não são avaliados como funções.

Regra de avaliação de `&&`

- Avalie a expressão à esquerda, se o valor for **False**, substitua toda a expressão por **False**;
- Senão, substitua toda a expressão pela expressão à direita.

Regra de avaliação de `||`

- Avalie a expressão à esquerda, se o valor for **True**, substitua toda a expressão por **True**;
- Senão, substitua toda a expressão pela expressão à direita.

O **echo** imprime a expressão que vem depois dele, precedida do arquivo e da linha onde aparece, e produz o valor dessa expressão como resultado.

Nos exemplos a seguir usamos esse efeito colateral para observar quais expressões são de fato avaliadas.

```
> 3 > 5 && { echo "aqui" True }
```

```
False
```

```
> 5 > 3 && { echo "aqui" True }
```

```
<repl>:1
```

```
"aqui"
```

```
True
```

```
> 3 > 5 || { echo "aqui" False }
```

```
<repl>:1
```

```
"aqui"
```

```
False
```

```
> 5 > 3 || { echo "aqui" True }
```

```
True
```

Igualdade

A linguagem Gleam oferece apenas um operador de igualdade, o `==`, que pode ser usado para quaisquer dois valores do mesmo tipo.

Em Gleam, dois valores são iguais se eles são estruturalmente iguais, isto é, se têm a mesma forma e as mesmas partes.

Os exemplos a seguir usam **listas**, que vamos estudar mais adiante — por enquanto basta saber que `[1, 2]` é uma lista com os elementos `1` e `2`, e que `[]` é a lista vazia.

O operador de diferente (negação da igualdade) é `!=`.


```
> 10 == 9 + 1
```

```
True
```

```
> 3.0 +. 1.0 == 4.0
```

```
True
```

```
> 10 == 10.0
```

```
error: Type mismatch
```

```
> ["a", "c", "b"] == ["a", "c", "b"]
```

```
True
```

```
> [[], [1, 2]] == [[], [1, 2]]
```

```
True
```

```
> [[], [1, 2]] != [[], [1, 2]]
```

```
False
```

Estilo de código

Você percebeu alguma diferença no estilo do código em relação ao que você está acostumado?

- A indentação é de dois espaços.
- Os nomes de tipos começam com maiúscula; os de funções e variáveis, com minúscula e em `snake_case`.

Em muitas linguagens isso é questão de gosto pessoal, em Gleam não.

Os nomes são impostos pelo compilador: `fn Soma(a, b)` e `fn somaQuadrados(a, b)` não compilam.

A formatação é definida pelo formatador:

```
$ sgleam format arquivo.gleam
```

No Sgleam Web, o formatador é sempre executado antes da execução do programa.

Convenção definida por ferramenta é melhor do que gosto pessoal: acaba a discussão e todo código tem a mesma aparência, o que facilita a leitura.

Revisão

O que é uma expressão?

- Um literal, uma função primitiva, um nome, uma forma especial ou uma chamada de função (uma expressão seguida de uma lista de expressões entre parênteses).

Por que dizemos que a definição de expressão tem **autorreferência**?

- Porque a definição refere-se a si mesma: uma chamada de função é formada por outras expressões. É isso que permite escrever expressões de tamanho arbitrário.

Por que dizemos que a regra de avaliação de expressões é **recursiva**?

- Porque para avaliar uma chamada de função é preciso avaliar as expressões que ela contém, isto é, a regra é descrita em termos dela mesma.

Quais tipos primitivos usamos até agora e quais são algumas de suas operações?

- `Int`, `Float`, `String` e `Bool`, com operações como `+` (`int.add`), `*` (`float.multiply`), `<>` (`string.append`) e `!` (`bool.negate`).

Por que `2 +. 3.0` não é uma expressão válida?

- Porque os operadores não são sobrecarregados: `+.` opera sobre dois `Float` e `2` é um `Int`.

Em uma chamada de função, o que é o operador e o que são os operandos?

- O operador é a primeira expressão, aquela que produz a função a ser aplicada; os operandos são as expressões entre parênteses. O valor da chamada é o resultado da aplicação do valor do operador aos valores dos operandos.

Para que servem as definições e onde ficam armazenados os nomes definidos?

- Servem para dar nome a objetos computacionais, sejam dados (**const**) ou funções (**fn**). Os nomes e os objetos associados a eles ficam em uma memória chamada **ambiente**.

O que é o modelo de substituição?

- É a forma de calcular o resultado da chamada de uma função composta: avalia-se o corpo da função substituindo cada ocorrência de um parâmetro pelo argumento correspondente.

Qual a diferença entre avaliação em ordem aplicativa e em ordem normal? Qual delas o Gleam usa?

- Na ordem aplicativa os argumentos são avaliados antes da substituição; na ordem normal a substituição é feita primeiro e os argumentos são avaliados apenas quando (e se) forem necessários. O Gleam usa a ordem aplicativa.

O que é uma **forma especial**?

- É uma construção que não é avaliada como uma chamada de função: ela tem uma regra de avaliação própria. O **case**, o **&&** e o **| |** são formas especiais.

Qual a diferença entre o **case** do Gleam e o **if** da maioria das linguagens?

- O **case** é uma expressão, ele produz um valor. Na maioria das outras linguagens o **if** é uma sentença: não produz valor, apenas seleciona quais comandos — e portanto quais efeitos colaterais — serão executados.

O que é uma definição local e qual é a sua regra de avaliação?

- É um nome introduzido com **let**, que vale do ponto em que é definido até o fim do bloco. Ele equivale a chamar uma função auxiliar cujo parâmetro é esse nome e cujo corpo é o resto do bloco, tendo como argumento a expressão à direita do **=**.

O que é avaliação em curto-circuito e por que ela não é obtida definindo **and** e **or** como funções?

- É deixar de avaliar o segundo operando quando o primeiro já determina o resultado. Com funções isso não acontece porque, na ordem aplicativa, todos os argumentos são avaliados antes da chamada. Por isso `&&` e `||` são formas especiais, com regra de avaliação própria.

Quando dois valores são iguais em Gleam?

- Quando eles são estruturalmente iguais. O `==` pode ser usado com dois valores de um mesmo tipo; comparar tipos diferentes é um erro.

Referências

Básicas

- Seção 1.1 - The Elements of Programming do livro SICP
- Tour da linguagem Gleam
- Biblioteca padrão do Gleam